In [1]:
```
# ================================
# 1. Import Libraries
# ================================
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from tensorflow.keras.datasets import cifar10
from tensorflow.keras import layers, models

from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

# ================================
# 2. Load Dataset
# ================================
(X_train, y_train), (X_test, y_test) = cifar10.load_data()

class_names = ['airplane','automobile','bird','cat','deer',
               'dog','frog','horse','ship','truck']

# Normalize images
X_train = X_train / 255.0
X_test = X_test / 255.0

y_train = y_train.flatten()
y_test = y_test.flatten()

# ================================
# 3. Visualize Sample Images
# ================================
plt.figure(figsize=(10,5))

for i in range(10):
    plt.subplot(2,5,i+1)
    plt.imshow(X_train[i])
    plt.title(class_names[y_train[i]])
    plt.axis('off')

plt.suptitle("Sample CIFAR-10 Images")
plt.show()

# ================================
# 4. Build CNN Model
# ================================
model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(32,32,3)),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(2,2),

    layers.Conv2D(128, (3,3), activation='relu'),

    layers.Flatten(),
```

```
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

# ================================
# 5. Train Model
# ================================
history = model.fit(
    X_train, y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.2
)

# ================================
# 6. Plot Accuracy & Loss Curves
# ================================
plt.figure(figsize=(12,5))

# Accuracy
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title("Accuracy Curve")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()

# Loss
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Loss Curve")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()

plt.show()

# ================================
# 7. Predictions
# ================================
y_pred_probs = model.predict(X_test)
y_pred = np.argmax(y_pred_probs, axis=1)

# ================================
# 8. Confusion Matrix
# ================================
cm = confusion_matrix(y_test, y_pred)
```

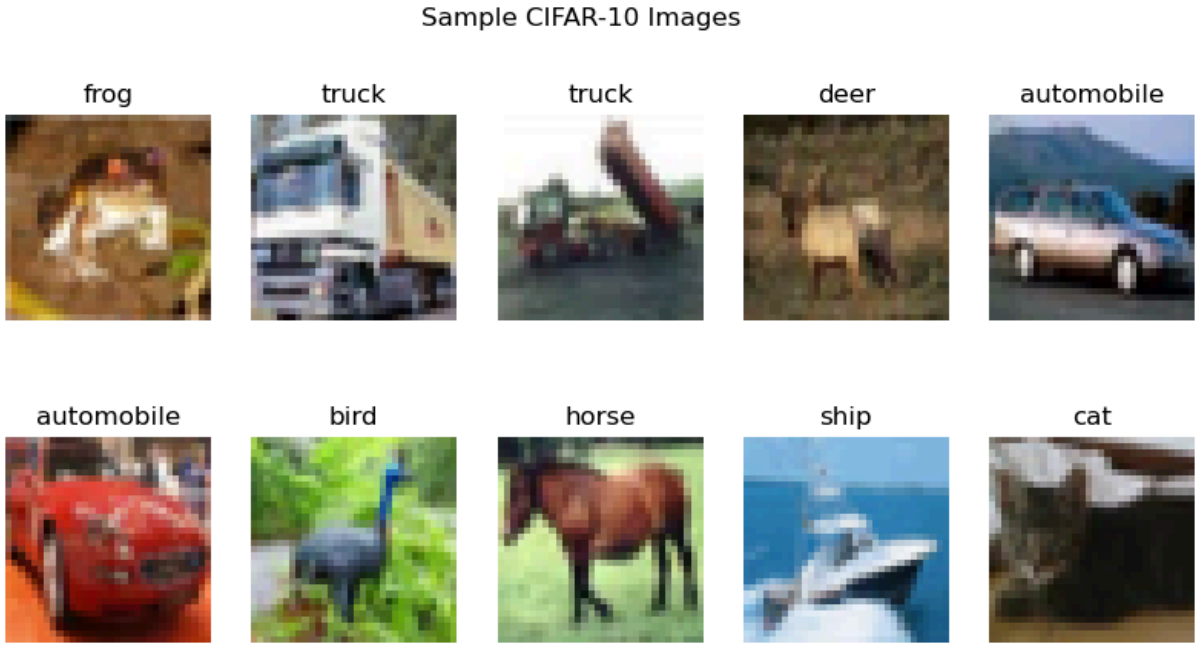
```
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names,
            yticklabels=class_names)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - CIFAR-10 CNN")
plt.show()

# ================================
# 9. Metrics Evaluation
# ================================
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("Precision (weighted):", precision_score(y_test, y_pred, average='weighted'))
print("Recall (weighted):", recall_score(y_test, y_pred, average='weighted'))
print("F1 Score (weighted):", f1_score(y_test, y_pred, average='weighted'))

print("\nClassification Report:\n")
print(classification_report(y_test, y_pred, target_names=class_names))
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 82s 0us/step
```




```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for Ten
sorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Please use
WSL2 or the TensorFlow-DirectML plugin.
```

```
C:\Users\admin\anaconda3\Lib\site-packages\keras\src\layers\convolutional\base_conv.
py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. W
hen using Sequential models, prefer using an `Input(shape)` object as the first laye
r in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
Model: "sequential"
```

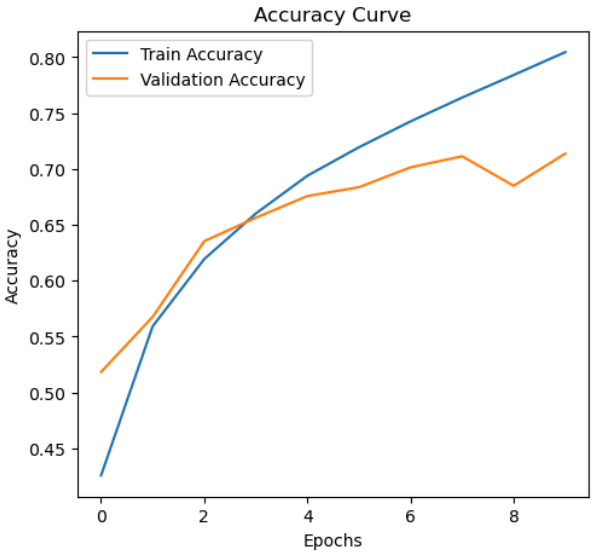
Layer (type)	Output Shape	
conv2d (Conv2D)	(None, 30, 30, 32)	
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	
conv2d_1 (Conv2D)	(None, 13, 13, 64)	
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	
conv2d_2 (Conv2D)	(None, 4, 4, 128)	
flatten (Flatten)	(None, 2048)	
dense (Dense)	(None, 128)	
dense_1 (Dense)	(None, 10)	

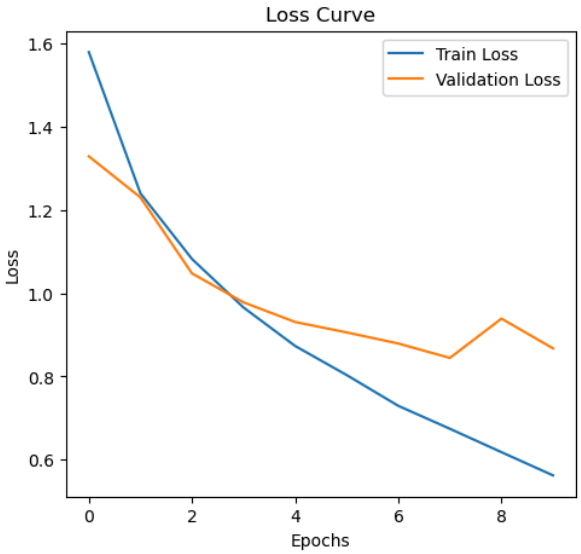
Total params: 356,810 (1.36 MB)

Trainable params: 356,810 (1.36 MB)

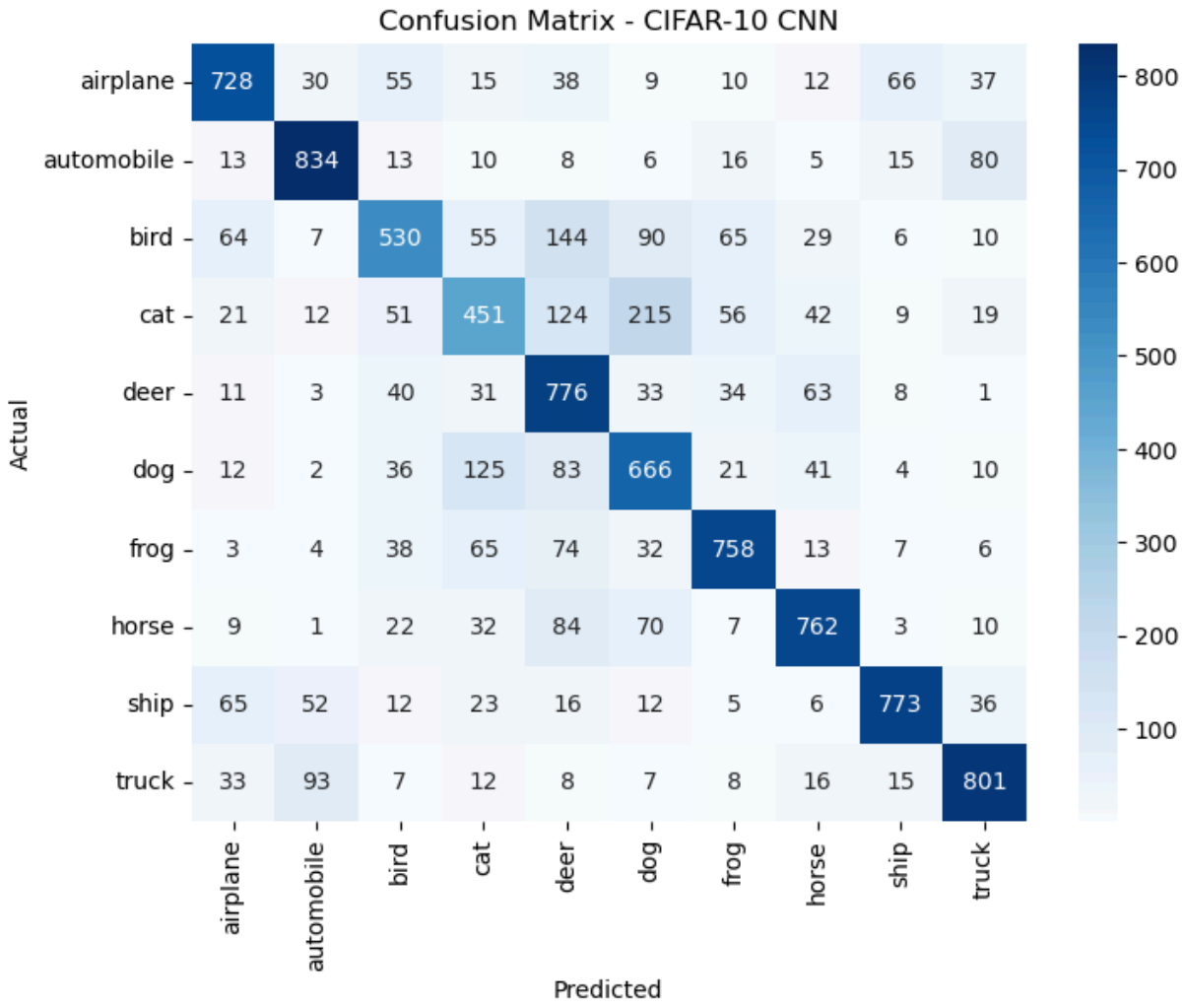
Non-trainable params: 0 (0.00 B)

```
Epoch 1/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 11s 16ms/step - accuracy: 0.4258 - loss: 1.5788 - val_a
ccuracy: 0.5184 - val_loss: 1.3286
Epoch 2/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 13s 21ms/step - accuracy: 0.5592 - loss: 1.2400 - val_a
ccuracy: 0.5677 - val_loss: 1.2300
Epoch 3/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 13s 21ms/step - accuracy: 0.6194 - loss: 1.0822 - val_a
ccuracy: 0.6354 - val_loss: 1.0474
Epoch 4/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 13s 20ms/step - accuracy: 0.6602 - loss: 0.9656 - val_a
ccuracy: 0.6564 - val_loss: 0.9780
Epoch 5/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 13s 21ms/step - accuracy: 0.6940 - loss: 0.8733 - val_a
ccuracy: 0.6758 - val_loss: 0.9309
Epoch 6/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 13s 20ms/step - accuracy: 0.7195 - loss: 0.8035 - val_a
ccuracy: 0.6837 - val_loss: 0.9058
Epoch 7/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 12s 20ms/step - accuracy: 0.7426 - loss: 0.7296 - val_a
ccuracy: 0.7015 - val_loss: 0.8792
Epoch 8/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 14s 22ms/step - accuracy: 0.7639 - loss: 0.6745 - val_a
ccuracy: 0.7114 - val_loss: 0.8446
Epoch 9/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 11s 18ms/step - accuracy: 0.7841 - loss: 0.6184 - val_a
ccuracy: 0.6850 - val_loss: 0.9390
Epoch 10/10
625/625 ━━━━━━━━━━━━━━━━━━━━ 13s 20ms/step - accuracy: 0.8046 - loss: 0.5626 - val_a
ccuracy: 0.7138 - val_loss: 0.8676
```






```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step
```



```
Accuracy: 0.7079
Precision (weighted): 0.7119586884898368
Recall (weighted): 0.7079
F1 Score (weighted): 0.7066607509617757

Classification Report:

              precision    recall  f1-score   support

    airplane       0.76      0.73      0.74      1000
  automobile       0.80      0.83      0.82      1000
        bird       0.66      0.53      0.59      1000
         cat       0.55      0.45      0.50      1000
        deer       0.57      0.78      0.66      1000
         dog       0.58      0.67      0.62      1000
        frog       0.77      0.76      0.77      1000
       horse       0.77      0.76      0.77      1000
        ship       0.85      0.77      0.81      1000
       truck       0.79      0.80      0.80      1000

    accuracy                           0.71     10000
   macro avg       0.71      0.71      0.71     10000
weighted avg       0.71      0.71      0.71     10000
```

In [ ]: